PY101 - Introduction to Python

Week 1: Core Foundations and Functions

Lesson 1: Recap: Printing and Comments

The `print()` function outputs text to the console, automatically adding a space between items separated by commas and a newline at the end. Use # to add single-line comments that Python will ignore.

```
print("age:", 25)
# Output: age: 25
```

Lesson 2: Variables, Types, and Strings

Variables store data and their values can be reassigned. Use `type()` to identify if a value is an `int`, `float`, or `str`.

- **TypeError:** Occurs if you try to mathematically add a string and a number directly.
- **NameError:** Occurs if you try to use a variable before defining it.
- **f-strings:** The cleanest way to mix variables and text (e.g., `f"{name} scored {score}%"`).

Lesson 3: Expressions and Operator Precedence

Python follows standard math order of operations (PEMDAS). Multiplication and division happen before addition and subtraction. You can override this by grouping operations with parentheses `()`.

```
print(2 + 3 * 4)    # Output: 14
print((2 + 3) * 4) # Output: 20
```

Note: Division (`/`) in Python always returns a float (e.g., `6 / 2` results in `3.0`).

Lesson 4: Updating Values and Printing Expressions

Use shortcut operators to efficiently update variable values:

- `+=` adds to the current value (e.g., `count += 3` is the same as `count = count + 3`).
- `-=` subtracts from the current value.

Lesson 5: Modeling Real-World Calculations

Using named variables makes math calculations much clearer than using raw numbers. Proper naming conventions help explain *why* a calculation is taking place (e.g., `join_fee + monthly_total` is clearer than `25 + 90`).

Lesson 6: Division Modes (/, //, %)

Python has three distinct ways to handle division:

- **True Division (`/`):** Returns a float with decimals (e.g., `7 / 2 = 3.5`).
- **Floor Division (`//`):** Drops the decimal and returns just the whole number/integer (e.g., `7 // 2 = 3`).
- **Modulo (`%`):** Returns only the *remainder* of the division (e.g., `7 % 2 = 1`).

Lesson 7: Modulo in Practice

Modulo is highly useful for determining cycles or checking if a number is even or odd:

- **Parity:** `n % 2` will always return `0` for even numbers and `1` for odd numbers.
- **Cycles:** Modulo can determine positions in repeating sequences (e.g., `turn % total_players`).

Lesson 8: Rounding and Money Format

When working with currency, you often need exactly two decimal places.

- `round(value, 2)`**:** Mathematically changes the actual underlying value to two decimal places.
- `f"${value:.2f}"`**:** Changes only the *display* of the value to two decimal places, leaving the underlying float untouched.

Lesson 9: Getting Input from Users

The `input()` function pauses the script to wait for the user to type something. **Important:** Data collected via `input()` is *always* a string. You must cast it to perform math.

```
age_txt = input("Age? ")
age = int(age_txt) + 1
```

Lesson 10: Functions I - Define and Call

Functions group code into reusable blocks, preventing repetition. Define a function using the `def` keyword, followed by a name, parentheses (which hold optional parameters), and a colon `:`. The code inside must be indented.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Adam")
```

Lessons 11 & 12: Functions II & III - Inside the Function & Return

The `return` keyword is used to send a value back out of a function to be used later in the program.

- **Print vs. Return:** `print()` just displays text on the screen. `return` hands data back to the script.
- **Early Exit:** A function stops executing immediately after hitting a `return` statement. Any code below it inside the function is ignored.

Lesson 13: Scope and Local Variables

Variables created inside a function are **local**; they only exist inside that specific function and do not overwrite variables with the same name outside the function. Passing data via parameters is much safer than relying on or altering global variables.

Lesson 14: Python Errors & Tracebacks

A traceback is Python's way of telling you what went wrong. It shows the file, line number, and the specific error type at the very bottom.

- **TypeError:** e.g., combining a string and an integer.
- **NameError:** e.g., printing a variable that hasn't been assigned yet.
- **ValueError:** e.g., trying to run `int("apple")`.

Lesson 15: Introduction to Debugging

A "logic bug" occurs when the code runs without error messages but produces the wrong result. The standard debugging process:

1. Inject `print()` statements inside functions to trace variable values.
2. Identify where the logic goes wrong.
3. Remove or comment out the debug `print()` statements to clean up the final output.